

Last Updated: Jan 16, 2026

How to Pull a Remote Branch Locally in Git: Get Colleague's New Branch When Only Master Exists

Imagine this scenario: You're collaborating with a team on a Git repository. Your colleague has just created a new feature branch, made changes, and pushed it to the remote repository (e.g., GitHub, GitLab, or Bitbucket). However, when you check your local repository, you only see the ``master`` (or ``main``) branch—no sign of their new branch. How do you fetch that remote branch to your local machine so you can review their code, test it, or contribute to it?

This is a common situation in team workflows, and Git provides straightforward tools to resolve it. In this guide, we'll walk through the step-by-step process to **fetch, list, and checkout a remote branch locally**, even if your local repo initially only has ``master``. We'll also cover troubleshooting tips and best practices to avoid common pitfalls.



Continue to the Next Step

Continue to the Next Step Continue to the Next Step



Continue to the Next Step

Continue to the Next Step



Continue to

Table of Contents

1. [Prerequisites](#)
2. [Step 1: Ensure Your Local Repository is Up-to-Date](#)
3. [Step 2: Fetch Remote Branches](#)
4. [Step 3: List Remote Branches to Confirm the New Branch Exists](#)
5. [Step 4: Checkout the Remote Branch Locally](#)
6. [Step 5: Verify the Local Branch is Tracking the Remote](#)
7. [Troubleshooting Common Issues](#)
8. [Best Practices for Collaborating with Remote Branches](#)
9. [Conclusion](#)
10. [References](#)

Prerequisites



- **Local Clone of the Repository:** You must have a local copy of the team's repository (created via ``git clone <remote-url>``).
- **Access to the Remote Repository:** You need permission to view and pull from the remote repo (e.g., you're added as a collaborator on GitHub).
- **Colleague's Branch Name:** Know the exact name of the branch your colleague pushed (e.g., ``feature/user-authentication``).

Step 1: Ensure Your Local Repository is Up-to-Date

First, navigate to your local repository directory using the terminal:

```
cd /path/to/your/local/repo
```

Before fetching new branches, it's good practice to ensure your local ``master`` branch is up-to-date (though this isn't strictly required for fetching remote branches). Run:

```
git checkout master  
git pull origin master
```

This updates your local ``master`` with the latest changes from the remote, but again, this step is optional for fetching *other* branches.

Step 2: Fetch Remote Branches



``git fetch`` downloads *metadata* about remote branches (e.g., new branches, updates to existing branches) to your local repository, but it does **not** merge these changes into your working directory. This is safer than ``git pull`` (which combines ``fetch`` and ``merge``), as it lets you inspect remote branches before integrating them.

Run ``git fetch``

To fetch all branches from the default remote (usually named ``origin``):

```
git fetch origin
```

If your team uses multiple remotes (e.g., ``upstream`` for the main repo and ``origin`` for your fork), fetch from all remotes with:

```
git fetch --all
```

You'll see output like this if new branches are found:

```
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 3), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (8/8), 1.20 KiB | 1.20 MiB/s, done.
From https://github.com/your-team/your-repo
* [new branch]      feature/user-authentication -> origin/feature/user-authentication
```



After fetching, list all remote branches to verify your colleague's branch is now visible locally.

List Remote Branches

Run:

```
git branch -r
```

The `-r` flag stands for "remote." You'll see a list like this:

```
origin/HEAD -> origin/master  
origin/master  
origin/feature/user-authentication # Your colleague's branch!
```

If the branch isn't listed, double-check:

- Did your colleague push the branch? Ask them to confirm with `git push origin <branch-name>`.
- Did you spell the remote name correctly? Use `git remote -v` to list remotes and confirm the URL is correct.

Step 4: Checkout the Remote Branch Locally

Now that the remote branch is fetched, you need to create a **local branch** that tracks (i.e., syncs with) the remote branch.

Option 1: Simplified Checkout (Git 2.23+)

If the remote branch name is `feature/user-authentication`, run:

```
git checkout feature/user-authentication
```

Or, using `git switch` (recommended for clarity):

```
git switch feature/user-authentication
```

Git will automatically create a local branch with the same name and set it to track `origin/feature/user-authentication`. You'll see:

```
Branch 'feature/user-authentication' set up to track remote branch 'feature/user-authentication'  
Switched to a new branch 'feature/user-authentication'
```

Option 2: Manual Branch Creation (Older Git Versions)

If you're using an older Git version (pre-2.23), or if you want to use a different local branch name (e.g., `my-user-auth` instead of `feature/user-authentication`), use `git checkout -b`:

```
git checkout -b <local-branch-name> origin/<remote-branch-name>
```

Example:



The `-b` flag creates a new local branch and checks it out. The `origin/<remote-branch-name>` specifies the remote branch to track.

Step 5: Verify the Local Branch is Tracking the Remote

To confirm your local branch is correctly tracking the remote branch, run:

```
git branch -vv
```

The `-vv` flag shows verbose branch info, including tracking status. You'll see output like:

```
* feature/user-authentication  a1b2c3d [origin/feature/user-authentication] Add login form
master                        e4f5g6h [origin/master] Update README
```

The `[origin/feature/user-authentication]` part confirms the local branch is tracking the remote.

Pull the Latest Changes (Optional)

Now that your local branch is set up, you can pull the latest commits from the remote branch (though `git fetch` should have already downloaded them):

```
git pull
```

Since the branch is tracking `origin/feature/user-authentication`, `git pull` will automatically fetch

Continue to the Next Step

Continue to the Next Step



Continue to

Troubleshooting Common Issues

Issue 1: Remote Branch Still Not Showing Up After `git fetch`

If `git branch -r` doesn't list your colleague's branch:

- **Colleague hasn't pushed the branch:** Ask them to run `git push origin <branch-name>`.
- **Typo in branch name:** Ensure you're using the exact name (branches are case-sensitive!).
- **Remote is misconfigured:** Check remotes with `git remote -v`. If the URL is incorrect, update it with `git remote set-url origin <correct-url>`.

Issue 2: "Fatal: 'origin/branch-name' is not a commit..."

If you see:

```
fatal: 'origin/feature/user-authentication' is not a commit and a branch 'feature/user-authenticat
```



- Confirm the remote branch exists on the remote repo (e.g., check GitHub's "Branches" tab).

Issue 3: Permission Denied

If ``git fetch`` fails with a permission error (e.g., ``remote: Permission to repo.git denied to user``), you likely lack access to the remote repo. Ask the repo admin to add you as a collaborator.

Best Practices for Collaborating with Remote Branches

1. **Fetch Regularly:** Run ``git fetch origin`` daily to stay updated on remote branches.
2. **Use Descriptive Branch Names:** Colleagues should use clear names (e.g., ``bugfix/login-error`` instead of ``fix123``) to avoid confusion.
3. **Communicate Branch Names:** Confirm the exact branch name with your colleague to avoid typos.
4. **Avoid Working Directly on `master`:** Always work on feature/bugfix branches, not ``master``, to keep the main branch clean.
5. **Update with ``git pull --rebase``:** When updating your local branch with remote changes, use ``git pull --rebase`` to avoid unnecessary merge commits.



Conclusion

Fetching and checking out a colleague's remote branch is a common Git workflow, and it only takes a few steps:

1. ``git fetch origin`` to download remote branch metadata.
2. ``git branch -r`` to confirm the remote branch exists.
3. ``git checkout <branch-name>`` (or ``git switch``) to create and track the local branch.

By following these steps, you'll seamlessly collaborate with your team and access new branches in no time. If you run into issues, double-check the branch name, fetch again, or verify remote permissions!

References



- [Atlassian Git Tutorials: Fetch](#)

📅 2025-12



